

all we need to do is to add MNT namespace to our script:

```
func run() {
    // Stuff we previously went over

    cmd.SysProcAttr = &syscall.
SysProcAttr{
        Cloneflags:  syscall.
CLONE_NEWUTS | syscall.CLONE_NEWPID |
syscall.CLONE_NEWNS,
        Unshareflags: syscall.CLONE_
NEWNS,
    }

    must(cmd.Run())
}
```

Now if we observe the mount points from our host OS with the container running by using *mount | grep proc* again, we can see that our container's mount point is invisible to our host OS.

We can now say that we have a truly isolated environment. We can also assign our container some arbitrary hostname, just so that there is a better distinction between our host and our containerised environments.

```
func child() {
    // Stuff that we previously went over

    must(syscall.Sethostname([]
byte("container")))
    must(syscall.Chroot("/home/
lubuntu/Projects/make-sense-of-
containers/lubuntu-fs"))
    must(syscall.Chdir("/"))
    must(syscall.Mount("proc", "proc",
"proc", 0, ""))

    must(cmd.Run())

    must(syscall.Unmount("/proc", 0))
}
```

Running it gives the output as can be seen in Figure 4.

```
root@lubuntu-vm:/home/lubuntu/Projects/make-sense-of-containers# go run main.go run bash
Running [bash] as PID 1
root@container:/#
```

Figure 4: Observing output of assigned hostname

This gives us a fully running, fully functioning container!

There is, however, one more important concept that we haven't covered yet. While namespaces provide isolation and layered file systems provide us with a root file system for our container, we need Cgroups for resource sharing.

Cgroups

Cgroups, also known as 'Control Groups' and previously referred to as 'Process Groups', are perhaps one of Google's most prominent contributions to the software world. Fundamentally, Cgroups collect a set of processes or task ids together and apply limits to them. While namespaces isolate processes, Cgroups enforce resource sharing between processes.

Just like */proc*, Cgroups too are exposed by the kernel as a special file system that we can mount. We add a process or thread to a Cgroup by simply adding process ids to a *tasks* file, and then read and configure various values by essentially editing files in that directory.

```
func cg() {
    // Location of the Cgroups
    filesystem
    cgroups := "/sys/fs/cgroup/"
    pids := filepath.Join(cgroups,
"pids")

    // Creating a directory named
'pratikms' inside '/sys/fs/cgroup/pids'
    // We will use this directory
to configure various parameters for
resource sharing by our container
    err := os.Mkdir(filepath.Join(pids,
"pratikms"), 0755)
    if err != nil && !os.IsExist(err) {
        panic(err)
    }
}
```

```
// Allow a maximum of 20 processes
to be run in our container
must(ioutil.WriteFile(filepath.
Join(pids, "pratikms/pids.max"), []
byte("20"), 0700))
```

```
// Remove the new cgroup after
container exits
```

```
must(ioutil.WriteFile(filepath.
Join(pids, "pratikms/notify_on_
release"), []byte("1"), 0700))
```

```
// Add our current PID to cgroup
processes
must(ioutil.WriteFile(filepath.
Join(pids, "pratikms/cgroup.procs"), []
byte(strconv.Itoa(os.Getpid()), 0700))
}
```

```
func child() {
    fmt.Printf("Running %v as PID
%d\n", os.Args[2:], os.Getpid())
```

```
// Invoke cgroups
cg()
```

```
cmd := exec.Command(os.Args[2],
os.Args[3:]...)
// Stuff that we previously went
over
```

```
must(syscall.Unmount("/proc", 0))
}
```

On running our container, we can see the directory 'pratikms' created inside */sys/fs/cgroup* from our host. It has all the necessary files in it to control resource sharing within our container.

When we *cat pids.max* from our host, we can see in Figure 5 that our container is limited to running a maximum of 20 processes at a time. If we *cat pids.current*, we can see the number of processes currently running in our container. Now, we need to test the resource limitation that we applied